NetPref

Real-Time Network Performance & Congestion Predictor — Comprehensive System Guide

Domain: Computer Networks & Big Data | Stack: Kafka 3.9 • Spark 3.5 • MLlib • DuckDB • Streamlit | Target: Computer Science Students & Engineers

1. What is This Project Actually About?

In any modern computer network—from local Wi-Fi hotspots to enterprise clouds and telecommunication backbones—network congestion and performance degradation are inevitable. When traffic surges, router queue buffers fill up (a phenomenon known as **bufferbloat**). As buffers overflow, routers are forced to drop packets (**packet loss**), and packet delivery times fluctuate violently (**high jitter**).

To human users and applications, this manifests as frustrating video call freezing, robotic audio, gaming latency spikes, and stalled file transfers.

❌ The Traditional Reactive Approach

Traditional network monitoring tools calculate packet loss and jitter *after* a flow has completely finished, or by continuously sending active probe packets (ping / traceroute) that consume network bandwidth and react only after user experience is already ruined.

✅ NetPref's Predictive Approach

NetPref observes basic, easily obtainable transport-layer behavior (connection duration, transfer rates, packet sizes, and TCP handshake timings) and uses a trained Machine Learning model to **predict whether a flow is degraded in real time**—without ever directly measuring packet loss or jitter!

> **The Highway Analogy (CS Intuition):** Imagine monitoring highway traffic. *Traditional monitoring* counts how many cars crashed or came to a dead halt 10 minutes ago. *NetPref* watches the subtle patterns—sudden speed drops, rapid lane weaving, and tailgating—to predict that traffic flow is degrading *before* the highway reaches total gridlock.

The Core Research Question

"Can a machine learning model accurately identify degraded network flows purely from flow behavioral signatures (duration, packet counts, inter-arrival times, window sizes, and round-trip handshake latency) without target leakage?"

NetPref proves this concept empirically on real-world network traces (UNSW-NB15), achieving **97.0% classification accuracy** and an **AUC of 0.994** while processing continuous live streams at hundreds of events per second.

2. Architecture & Technology Stack: What is Used for What?

NetPref combines the standard enterprise Big Data and distributed streaming stack:

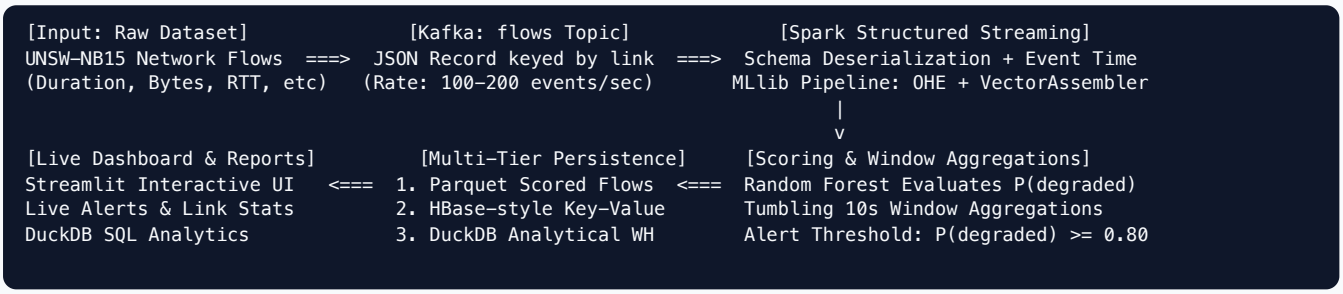
Technology	Architectural Layer	Specific Purpose & Engineering Justification
Apache Kafka 3.9 `(KRaft Mode)`	Message Ingestion Buffer	Decouples high-throughput traffic generation from downstream stream processing. Uses KRaft (Kafka Raft consensus, no ZooKeeper needed) for lightweight execution. Events are partitioned by `link_id` so traffic per link maintains strict sequential order.
Apache Spark 3.5 `(PySpark Streaming)`	Distributed Stream Processing	Spark Structured Streaming processes events in micro-batches (every 5 seconds). It scales horizontally, handles backpressure, deserializes JSON, watermarks out-of-order data, and applies stateful tumbling window aggregations.
Spark MLlib `(Random Forest)`	Machine Learning Inference	Applies a pre-trained Random Forest (50 trees, max depth 8) inside the streaming DAG. Because MLlib runs natively inside the Spark JVM, it scores thousands of flows per second in parallel without Python-IPC bottlenecks.
HBase-Style Store `(HBaseLite over DuckDB)`	Operational Key-Value Store	Emulates an Apache HBase wide-column store with compound row keys (`link_id#reversed_ts#flow_id`) and column families

Technology	Architectural Layer	Specific Purpose & Engineering Justification
		(`flow:` , `pred:` , `alert:`). Reversed timestamps ensure that a prefix scan of a link returns the newest events first.
DuckDB Warehouse `(warehouse.duckdb)`	Analytical Data Warehouse	High-speed columnar OLAP engine for SQL reporting. Stores the `fact_flows` table and `v_alerts` view, powering slicing-and-dicing by protocol, service, time, and link health without requiring a heavy DBMS server.
Streamlit & Altair `(dashboard.py)`	Interactive Visual NOC	Web dashboard displaying real-time link health cards, alert feeds, model quality comparisons, interactive Altair charts, and an in-browser SQL query runner for deep exploration.

3. Input → Process → Output: Deep Dive

To truly understand the system, trace the journey of a single network flow record from raw data to an executive alert:

The End-to-End Data Lifecycle



A. What is the Input?

The system ingests records from the **UNSW-NB15** benchmark network dataset. In computer networks, a *flow* is a sequence of packets between a source IP and destination IP using a specific transport protocol and ports (the 5-tuple).

Each input record contains **36 network metrics**, including:

- **General Flow Statistics:** `dur` (connection duration in seconds), `spkts` / `dpkts` (packets sent by source/destination), `sbytes` / `dbytes` (bytes sent), `rate` (overall transfer rate).
- **Timing & Dynamics:** `sinpkt` / `dinpkt` (mean inter-packet arrival time in milliseconds), `smean` / `dmean` (average packet size in bytes).
- **TCP Handshake Metrics:** `tcprtt` (round-trip time), `synack` (SYN-to-SYN-ACK latency), `ackdat` (SYN-ACK-to-ACK latency), `swin` / `dwin` (TCP window advertisements).

> **Critical Engineering Safeguard (Zero Target Leakage):** The training script (`prepare_data.py`) derives the ground-truth "degraded" label using loss ratio (packet drops > 85th percentile) and jitter (> 90th percentile). However, **these loss and jitter columns are strictly excluded from model features**! The model never sees them; it must predict degradation solely from behavioral characteristics like timing, rates, and window sizes.

B. What is the Process?

1. **Replay & Serialization (`producer.py`):** Replays test flows into Kafka topic `flows` . Each flow is tagged with a simulated link ID (`link-1` to `link-8`) and stamped with `event_ts_ms` (epoch milliseconds).
2. **Streaming Consumption (`stream_job.py`):** Spark subscribes to Kafka with `maxOffsetsPerTrigger=500` , deserializing the JSON string into strongly-typed Spark DataFrame columns and generating an `event_time` timestamp.
3. **ML Inference Pipeline:** Features pass through `StringIndexer` & `OneHotEncoder` (for categorical columns `proto` , `service` , `state`), assembled into a dense vector with `VectorAssembler` , and evaluated by the 50-tree Random Forest.
4. **Dual Stream Outputs:**
    - *Stream 1 (Scored Flows):* Every flow is enriched with `prediction` (0 or 1) and `degraded_prob` (0.0 to 1.0), written as append-only Parquet.

- *Stream 2 (Windowed Link Aggregates):* A 10-second tumbling window groups traffic by `(window, link_id)` with a 30-second watermark, calculating average loss, mean jitter, and degraded flow rate.

5. **HBase Key-Value Indexing (`hbase_layer.py`):** Structures data for instant sub-millisecond retrieval using a compound row key:

   `link_id` # `(9999999999999 - timestamp_ms)` # `flow_id`

   Because lexicographical sorting arranges smaller numbers first, **reversing the timestamp guarantees that the newest flow on any link always sorts to the top**!

C. What is the Output to Understand?

- **Degraded Probability (P(degraded)):** The calibrated probability that this flow is suffering degraded performance.
- **Real-Time Alert Feed:** Flows where P(degraded) ≥ 0.80 trigger alerts, alerting network operators before catastrophic buffer overflow.
- **Per-Link Degradation Ratio (%):** Percentage of active traffic flagged as degraded on each link, immediately identifying bottleneck transit routes.
- **Confusion Matrix & Accuracy:** Verified empirical validation against the held-out test set (97.0% Accuracy, 95.2% Precision, 92.6% Recall).

4. How to Run & Give a Live Demonstration (With Visuals)

Follow this exact step-by-step procedure to execute NetPref locally and deliver a compelling, professional technical demo.

Step 1: Environment & Prerequisites

Ensure you have Python 3.11, Java 17, and Apache Kafka installed. Activate your virtual environment:

```
source .venv/bin/activate
pip install -r requirements.txt
python test_spark.py          # Verifies PySpark and Java 17 connectivity
```

Step 2: One-Time Data Prep & Model Training

```
python prepare_data.py        # Cleans dataset, calculates quantile thresholds -> data/
python train_model.py         # Trains Spark MLlib Random Forest -> models/rf_congestion/
```

Step 3: Launch the 4 Components in Separate Terminals

Terminal	Command	Expected Output / Ready State
Terminal 1: Kafka	`kafka-server-start .../server.properties`	Kafka broker listens on `localhost:9092`.
Terminal 2: Spark Job	`python stream_job.py --seconds 300`	Prints `Streaming started... Waiting for Kafka events`.
Terminal 3: Dashboard	`streamlit run dashboard.py`	Opens browser at `http://localhost:8501` (Live monitor tab).
Terminal 4: Producer	`python producer.py --rate 100 --limit 2000`	Streams 2,000 flows at 100 flows/sec. Micro-batches begin scoring!

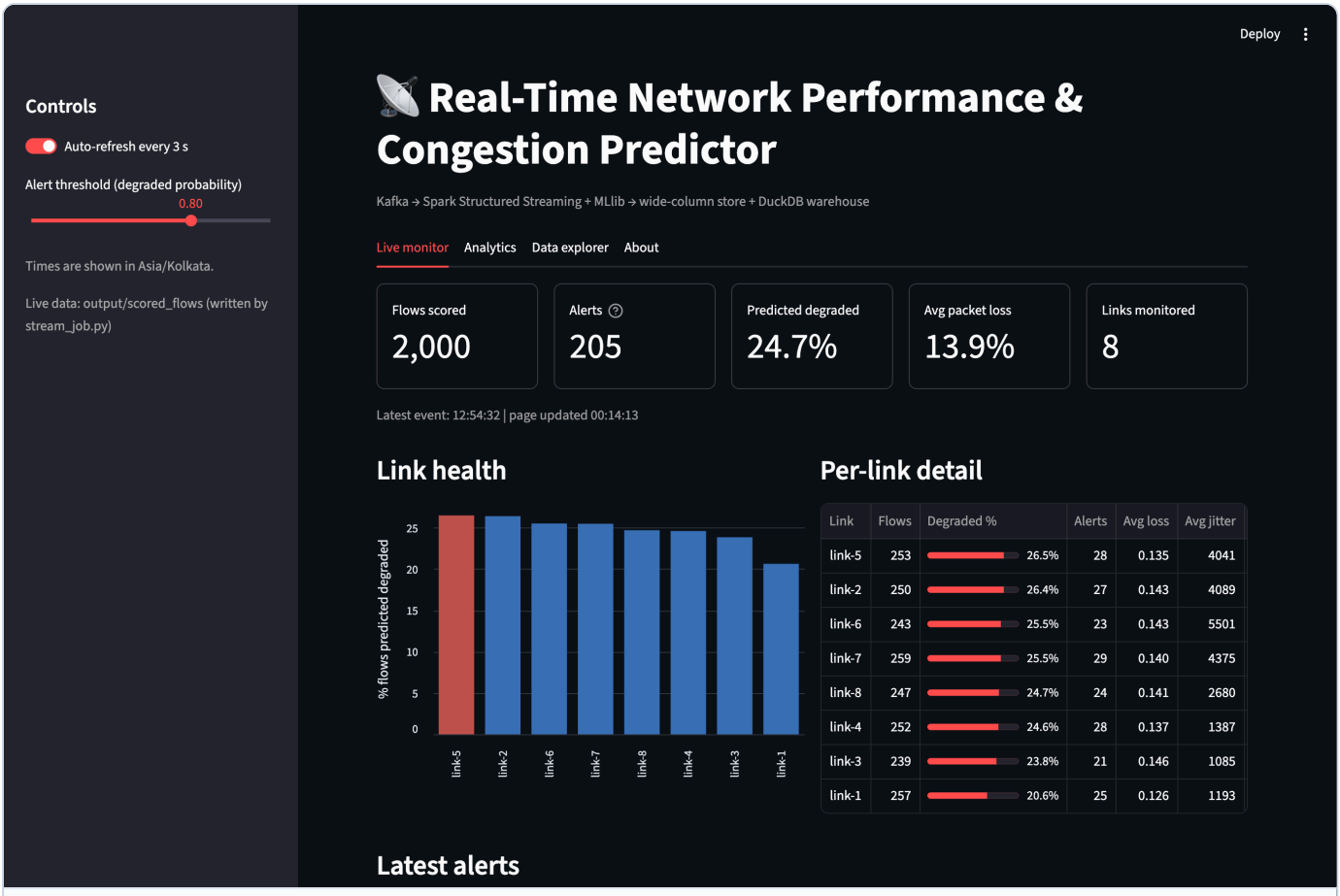


Figure 1: Authentic Live Streamlit Monitoring Cockpit. Displays real-time metric cards (2,000 scored flows, 205 alerts, 24.7% degraded rate), per-link health bar charts with worst-link highlighting (red), tabular progress meters, and latest alert feed.

Step 4: Load Analytical Stores (After Streaming Finishes)

```
python hbase_layer.py        # Ingests scored flows into HBaseLite wide-column store
python warehouse.py          # Builds fact_flows in DuckDB warehouse and generates charts
python show_outputs.py       # Summarizes all storage locations, sizes, and record counts
```

Analytical Warehouse Charts (Generated by DuckDB & Matplotlib)

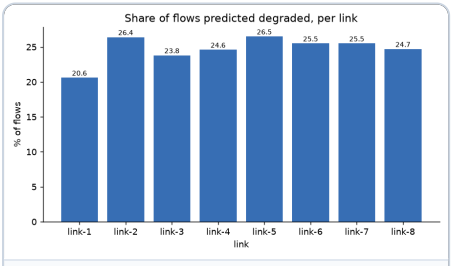


Fig 2: Share per Link. Visualizes degraded ratio across all 8 monitored links.

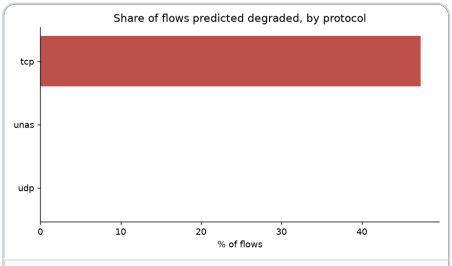


Fig 3: By Protocol. Compares degradation across TCP, UDP, and routing protocols.

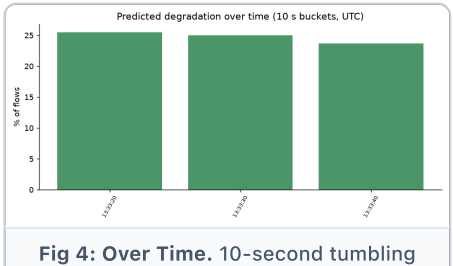


Fig 4: Over Time. 10-second tumbling time buckets showing temporal congestion spikes.

🎙️ The 5-Minute Technical Pitch / Demonstration Script

Minute 1 — The Problem: *"In modern networking, packet loss and jitter kill user experience. But measuring them directly is reactive. NetPref predicts degradation in real time using only standard flow behaviors."*

Minute 2 — The Live Stream: Point to **Figure 1**: *"Here is our live pipeline. Producer pushes events into Kafka. Spark Structured Streaming ingests 500 events per micro-batch, applies our Random Forest model, and flags flows with P(degraded) ≥ 0.80 as critical alerts."*

Minute 3 — Per-Link Telemetry: Show the link health cards: *"Notice how each link is continuously monitored. In the detail table, we see flows, loss ratio, average jitter, and total megabytes. Link-5 is automatically highlighted in red because its degradation rate spiked to 26.5%."*

Minute 4 — Big Data Storage: Click the **Data Explorer** tab: *"We use a dual-store architecture. High-speed operational lookups go to our HBase-style wide-column store where row keys use reversed timestamps for fast chronological scans. Historical SQL analytics query our DuckDB warehouse."*

Minute 5 — Model Performance: Click the **Analytics** tab: *"Our model scores 97.0% accuracy and 0.994 AUC. The top predictive features are connection duration, packet counts, TCP window sizes, and handshake latency—proving that network degradation leaves unmistakable behavioral fingerprints."*

5. How This Project Helps & Where Does It Actually Help?

NetPref addresses real-world operational challenges across multiple telecommunications and computer systems domains:

🌍 Internet Service Providers (ISPs) & Telecoms

ISPs manage thousands of transit links. NetPref enables **predictive traffic engineering**: when predictive degraded ratios rise on a trunk link, Software-Defined Network (SDN) controllers can automatically reroute non-critical traffic to secondary links *before* buffer overflow occurs.

☁️ Cloud Data Centers & Multi-Tenancy

In AWS/GCP virtualized networks, "noisy neighbor" virtual machines can flood shared virtual switches. NetPref identifies degraded flows and pinpoints which tenant flows are suffering latency collapse without invading payload privacy.

🎬 Real-Time Streaming & WebRTC (Zoom, Netflix)

Video conferencing algorithms adjust video bitrates based on packet loss. NetPref can signal impending degradation seconds ahead of time, allowing encoders to downscale resolution smoothly rather than freezing mid-call.

🛡️ Self-Healing & Autonomous Networks

Forms the telemetric foundation for AIOps (Artificial Intelligence for IT Operations). Predictive alerts can trigger automated Quality of Service (QoS) bandwidth shaping, rate-limiting, and firewall policy adjustments without human intervention.

Summary Takeaway: NetPref transforms network performance monitoring from a *post-mortem diagnostic chore* into a *proactive, real-time intelligence capability* using modern distributed big-data architecture.